

Appello TAP del 12/06/2026

Avete a disposizione **2 ore (2 ore e 30 minuti per chi ha DSA/disabilità)**.
La prova è composta da:

- **Quiz** su AulaWeb (11 punti)
- Esercizio di **programmazione** (11 punti)
- Esercizi di **testing** ($6 + 5 = 11$ punti)

La somma totale è **33 punti**, ma è necessario arrivare almeno a **5 punti su ciascuna delle tre parti** per passare l'esame.

Suggerimento: leggete **tutto** il testo prima di buttarvi a implementare “cose”: la specifica dei test dell'esercizio 2 può aiutarvi a capire fin da subito cosa implementare (e cosa **NON** fare).

Per la consegna, caricate su AulaWeb la **solution “zippata”**, ovvero zippate la cartella che contiene il file `.sln` e i progetti, *dopo aver eseguito il Clean*. Codice che non compila o che non passa il **SanityCheck** (presente anche in versione `.txt` su AulaWeb) non verrà considerato. Dopo aver caricato la solution, per sicurezza, scaricate il vostro zip, scompattatelo in un'altra cartella e verificate che compili tutto.

Esercizio 1 (11 punti)

In un progetto *class library* implementare l'extension method `Windowed<T>` che, dati:

- una sequenza (*potenzialmente infinita*) di elementi di tipo `T`, `source`
- un intero positivo `size`

restituisce, nell'ordine, tutte le *finestre scorrevoli* di lunghezza `size` contenute in `source`. Ogni finestra deve essere rappresentata come un array di `T`, cioè come un valore di tipo `T[]`.

Per esempio, se `source` fosse la sequenza `[1, 2, 3, 4]` e `size` fosse `3`, il risultato dovrebbe essere la sequenza contenente i due array:

```
[1, 2, 3]
[2, 3, 4]
```

Se `size == 1`, il risultato conterrà una finestra di un solo elemento per ogni elemento di `source`. Se `source` contiene meno di `size` elementi, il risultato dovrà essere la sequenza vuota.

Se `size <= 0`, il metodo dovrà lanciare `ArgumentOutOfRangeException` **immediatamente**, cioè senza che sia necessario enumerare la sequenza restituita.

Vincoli

- Enumerare la sequenza **source una sola volta**
- Il metodo deve funzionare anche con sequenze *potenzialmente infinite*
- Il metodo non deve enumerare elementi di **source** prima che qualcuno inizi a enumerare il risultato

Sanity check

Lo scopo di questo test è solo verificare che la *signature* del vostro extension method sia abbastanza simile a quanto atteso da permettere di compilare ed eseguire i test.

```
[Fact]
public void SanityCheck() {
    IEnumerable<int[]> x = Array.Empty<int>().Windowed(3);
    _ = new List<string> { "a", "b" }.Windowed(1);
    _ = new [] { 3.14, 2.71, 1.41 }.Windowed(2);
}
```

Esercizio 2 (6 punti)

Implementare, usando xUnit ed eventualmente Moq, i seguenti test relativi all'esercizio 1.

1. Casi “buoni”

- l'array vuoto [], con **size** = 3, deve restituire la sequenza vuota
- la lista di interi [1, 2, 3, 4], con **size** = 2, deve restituire le finestre [1, 2], [2, 3], [3, 4], in questo ordine
- la lista di interi [1, 2, 3], con **size** = 3, deve restituire una sequenza con una sola finestra, [1, 2, 3]
- l'array di interi [10, 20], con **size** = 3, deve restituire la sequenza vuota
- l'array di stringhe ["qui", "quo", "qua"], con **size** = 1, deve restituire le finestre ["qui"], ["quo"], ["qua"], in questo ordine

2. Eccezioni

- **size** = 0 deve sollevare **immediatamente** `ArgumentOutOfRangeException`
- **size** = -7 deve sollevare **immediatamente** `ArgumentOutOfRangeException`

Per “immediatamente” si intende che l'eccezione deve essere sollevata già invocando `source.Windowed(size)`, senza dover chiamare `ToList/ToArray`, usare `foreach` o altri metodi che enumerino la sequenza.

Esercizio 3 (5 punti)

Implementare uno o più test che verifichino il comportamento dell'extension method `Windowed<T>` rispetto a lazy evaluation e sequenze infinite.

In particolare, verificate che:

1. invocando `var windows = source.Windowed(2);` su una sequenza creata da voi, che incrementa un contatore quando viene enumerata, il contatore sia ancora 0 dopo la chiamata
2. dopo aver enumerato completamente `windows`, il contatore indichi che `source` è stata enumerata **una sola volta**
3. invocando `Windowed(3)` sulla sequenza infinita dei numeri interi non negativi `[0, 1, 2, 3, ...]` ed estraendo solo le prime quattro finestre, il risultato sia:

```
[0, 1, 2]  
[1, 2, 3]  
[2, 3, 4]  
[3, 4, 5]
```

In altre parole, il test sul caso infinito deve terminare e non deve tentare di materializzare l'intera sequenza sorgente.